



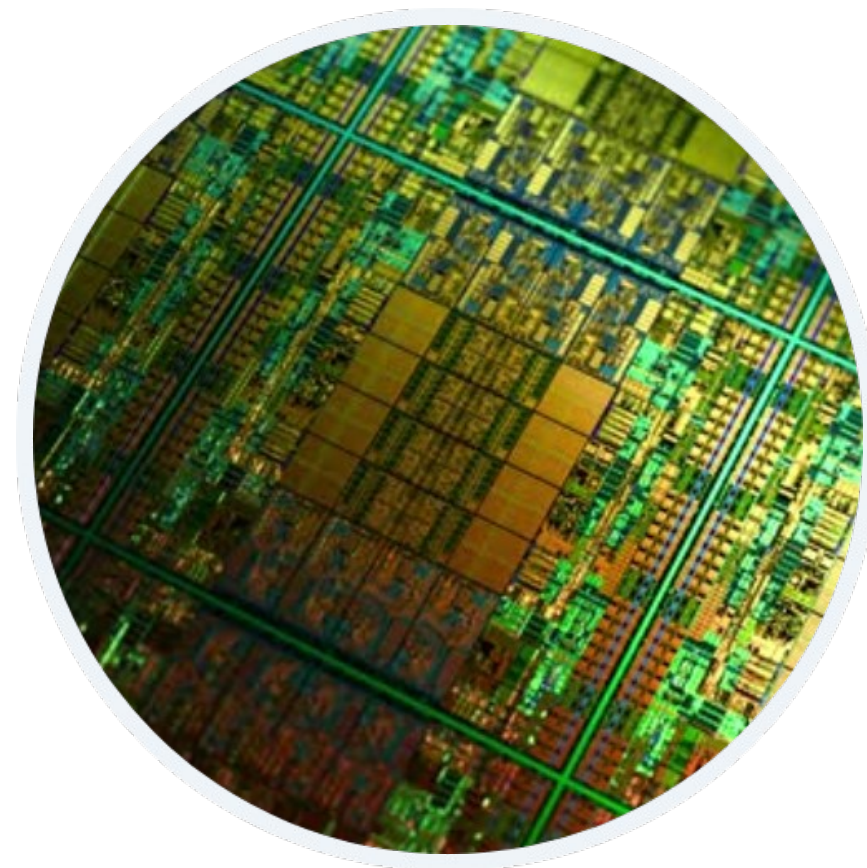
NEWTON SCHOOL OF AI & COMPUTING

COMPUTER ORGANIZATION

UNIT 1: INTRODUCTION AND PHYSICAL DATA HANDLING

Presented By: Bibrata Sarkar

Assistant Professor





SYLLABUS OVERVIEW OF UNIT 1

Part A: Architecture Fundamentals

- Computer Organization vs. Computer Architecture
- Basic Functional Units (Input, Output, Memory, ALU, CU)
- Von Neumann Architecture
- Harvard Architecture

Part B: Data Representation & Logic

- Number Systems (Binary, Octal, Decimal, Hexadecimal) and Binary Arithmetic (+, -), BCD, Gray Conversions
- Code, ASCII
- Logic Gates, Truth Tables, and Combinational Circuits (Adder, MUX, DEMUX)



INDUSTRY APPLICATIONS

Why do we study Computer Organization in the real world?

- **Microprocessor Design:** Designing efficient CPUs at Intel, AMD, Apple (M-series chips).
- **Embedded Systems:** Creating logic for smart home devices, automotive controls, and IoT sensors.
- **System Programming:** Writing low-level OS kernels and device drivers requires deep hardware knowledge.
- **Hardware Engineering:** Designing motherboards, physical memory buses, and logic controllers.
- **AI Hardware Acceleration:** Optimizing data pathways for GPUs and Neural Processing Units (NPUs).



COURSE OBJECTIVES

This course is designed to achieve the following overarching goals:



COURSE OUTCOMES (CO)

Upon successful completion of this course, students will be able to:

-
-
-
-
-



PROGRAM OUTCOMES (PO)

Computer Organization | Unit 1 | Course Outcome 1



CO – PO MAPPING

Computer Organization | Unit 1 | Course Outcome 1



PROGRAM SPECIFIC OUTCOMES (PSO)

Program Specific Outcomes for AI & Computer Science:

-
-
-



CO – PSO MAPPING



PROGRAM EDUCATIONAL OBJECTIVES

-
-
-



RESULT ANALYSIS

Historical Academic Performance in Computer Organization:

Computer Organization | Unit 1 | Course Outcome 1



QUESTION PAPER TEMPLATE



PREREQUISITE

Before beginning this course, students should ideally possess:

-
-
-
-

Computer Organization | Unit 1 | Course Outcome 1



INTRODUCTION TO COMPUTER ORGANIZATION

What exactly is Computer Organization?

- It is the study of the internal physical workings, structural arrangement, and hardware implementation of a computer system.
- While a software developer writes code, Computer Organization explains how the machine electrically executes that code bit-by-bit.
- It focuses heavily on physical data handling, memory interfaces, timing, and control signals generated by the processor.
- It forms the bridge connecting abstract computer architecture to raw electronic circuitry.



SUBJECT INTRODUCTION VIDEO RESOURCE



TOPIC LEARNING LINK

Title: Comprehensive Intro to Computer Org & Arch

Resource: NPTEL / Official Engineering Channel

[https://nptel.ac.in/courses/
106105163](https://nptel.ac.in/courses/106105163)

(Faculty Note: Please instruct students to watch this visualization module prior to the next lecture.)



UNIT 1 CONTENTS

- Org vs. Arch Definitions
- Basic Functional Units
- Von Neumann & Harvard
- Number Systems (Bin, Oct, Dec, Hex)
- Base Conversions
- Binary Arithmetic (+, -)

- Codes: BCD, Gray, ASCII
- Boolean Algebra (Axioms/Theorems)
- Logic Gates (Basic & Universal)
- Combinational Circuits Overview
- Half Adder & Full Adder
- Multiplexer (MUX) & Demultiplexer (DEMUX)



UNIT 1 OBJECTIVES

By the end of Unit 1, students will successfully:

- Differentiate clearly between computer organization (HOW) and architecture (WHAT).
- Identify the physical functional units of a modern PC and trace the data flow between them.
- Perform rapid, error-free base conversions and binary mathematical arithmetic.
- Understand how textual characters and numbers are physically represented via ASCII
- Simplify digital logic circuits using Boolean Algebra and Logic Gates to optimize hardware costs.



TOPIC 1: COMPUTER ORG VS ARCHITECTURE

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To clearly define and systematically contrast Computer Organization and Computer Architecture.

Topic Outcome: Students will be able to distinguish between the 'What' (Architecture) and the 'How' (Organization) of a computer system, explaining these concepts using real-world processor examples.



INTRODUCTION TO THE TOPIC

When designing a computer, engineers look at the system from two different perspectives:

Computer Architecture:

Seen by the programmer. It focuses on the logical aspects of system behavior.

Computer Organization:

Seen by the hardware designer. It focuses on the physical structure and electrical signals.



DEFINITIONS

Computer Architecture: Refers to those attributes of a system visible to a programmer, or those attributes that have a direct impact on the logical execution of a program. It is the conceptual design and fundamental operational structure.

Computer Organization: Refers to the operational units and their interconnections that realize the architectural specifications. It describes the physical hardware details that are transparent to the programmer.



DETAILED THEORY: WHAT VS HOW

The "WHAT" (Architecture):

Architecture dictates **what** the computer can do. For example, does the computer have an instruction for multiplication? What data formats are supported? What is the size of the memory address?

The "HOW" (Organization):

Organization dictates **how** the computer does it. For example, is the multiplication instruction physically implemented using a dedicated multiplier circuit, or is it done using repeated additions via a basic adder circuit?



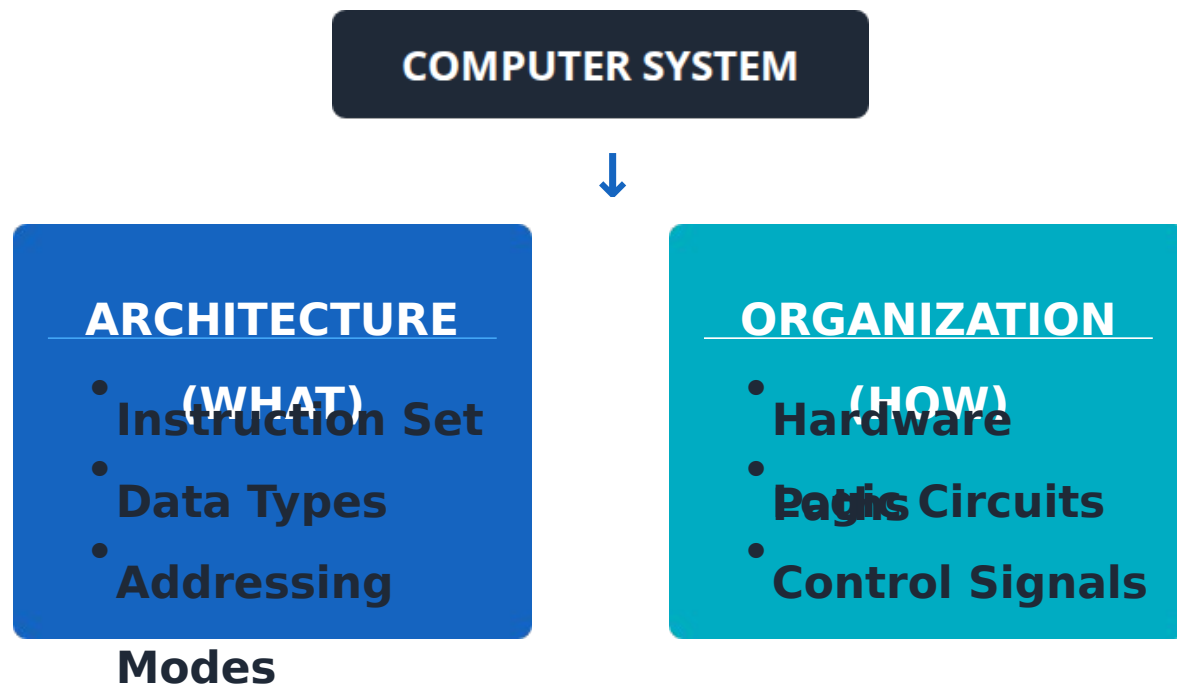
KEY CONCEPTS & PROCESSOR EXAMPLES

- **Instruction Set Architecture (ISA):** The vocabulary of the machine. Forms the boundary between software and hardware (Architecture).
- **Implementation Details:** Control signals, physical memory technology used (SRAM vs DRAM), computer bus design (Organization).

Processor Example:

Both Intel Core i7 and AMD Ryzen processors share the exact same *x86 Architecture*. They understand the same instructions. However, their internal *Organization* (how they physically route data, pipeline instructions, and arrange cache memory) is entirely different!

TECHNICAL DIAGRAM: SYSTEM HIERARCHY





STEP-BY-STEP EXPLANATION

Step 1: A software developer writes a program in C++.

Step 2: The compiler converts it into Assembly Language, bounded by the *Architecture's* Instruction Set.

Step 3: The CPU reads these instructions. *Organization* dictates how fast it reads them from RAM via physical buses.

Step 4: The Control Unit (part of Organization) sends electrical pulses to the ALU to execute the math.



REAL-LIFE ANALOGY

The Building Construction Analogy

Architecture = The Blueprint.

It dictates that the house will have 3 bedrooms, 2 doors, and 4 windows. It is the conceptual layout that the homeowner (programmer) cares about.

Organization = The Construction Phase.

It dictates the type of bricks used, the routing of the electrical copper wires inside the walls, and the brand of the plumbing pipes. The homeowner doesn't see this, but it determines how sturdy and efficient the house is physically.



COMPARISON TABLE

Feature	Computer Architecture	Computer Organization
Focus	System behavior & logical structure	Physical implementation & circuitry
Perspective	Software / Programmer View	Hardware / Engineer View
Core Question	WHAT does it do?	HOW does it do it?
Components	Instruction set, Registers, Formats	Control signals, Memory types, Buses
Evolution	Rarely changes (Backward compatible)	Changes frequently with technology



DAILY QUIZ & TOPIC LINK

DAILY QUIZ

Which of the following is an architectural attribute?

A. Memory technology used

B. Control signals generated

C. Instruction Set

D. Motherboard size

TOPIC LEARNING LINK

Video: Arch vs Org Explained

<https://youtu.be/XiF1onhCy8M?si=qfs9Urc5biYQyMg0>



POP QUIZ: TEST YOUR KNOWLEDGE

QUESTION 1

Which unit is responsible for coordinating all activities within the computer system?

- A. Arithmetic Logic Unit (ALU)
- B. Input Unit
- C. **Control Unit (CU) ✓**
- D. Output Unit

QUESTION 2

Where are the instructions and intermediate data stored during execution?

- A. **Memory Unit ✓**
- B. Secondary Storage
- C. Control Unit
- D. Arithmetic Logic Unit

QUESTION 3

Which of the following serves as the primary gateway for user commands into the CPU?

- A. Output Unit
- B. **Input Unit ✓**
- C. Registers
- D. Accumulator

QUESTION 4

What physical unit performs mathematical computations and comparison operations?

- A. **ALU ✓**
- B. Control Unit
- C. System Bus
- D. RAM



TOPIC 2: BASIC FUNCTIONAL UNITS

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To understand the 5 basic physical functional units that make up every digital computer system.

Topic Outcome: Students will be able to draw the block diagram of a computer, explain the physical flow of data, and describe the precise role of ALU, CU, Memory, Input, and Output units.



INTRODUCTION TO FUNCTIONAL UNITS

A computer consists of five functionally independent main parts:

- Input Unit
- Memory Unit
- Arithmetic Logic Unit (ALU)
- Control Unit
- Output Unit

The ALU and Control Unit are jointly grouped into what we call the **Central Processing Unit (CPU)**.



DETAILED THEORY: I/O & MEMORY

Input Unit: Accepts information from human operators (or sensors) and converts it into computer-readable binary format (0s and 1s). *Examples: Keyboard, Mouse, Scanner.*

Output Unit: The exact reverse of the input unit. It takes processed binary data and converts it back into human-understandable format. *Examples: Monitor, Printer.*

Memory Unit: Stores programs and data.

- **Primary Memory (RAM):** Fast, volatile, directly accessed by the CPU. Stores active programs.
- **Secondary Memory (HDD/SSD):** Slower, massive capacity, non-volatile. Stores permanent data.



DETAILED THEORY: ALU & CONTROL UNIT

Arithmetic Logic Unit (ALU):

- The "muscle" of the computer.
- Performs all mathematical operations (Addition, Subtraction, Multiplication, Division).
- Performs logical operations (AND, OR, NOT comparisons).

Control Unit (CU):

- The "nervous system" of the computer.
- It does not execute data; it generates timing and control signals.
- It acts as a traffic cop, routing data between memory, ALU, and I/O units to execute instructions in the correct sequence.



KEY CONCEPT: REGISTERS

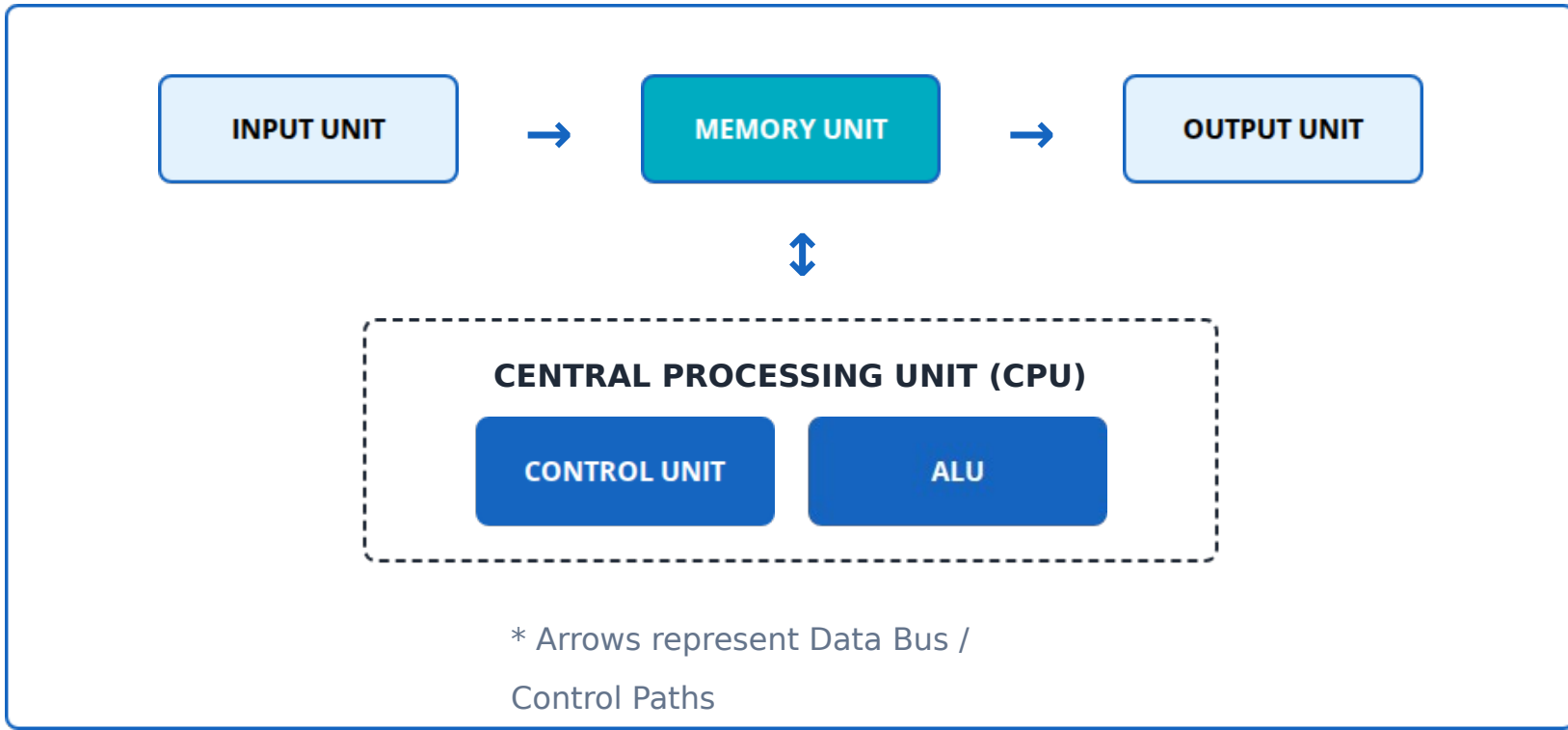
Where does data go immediately before the ALU processes it?

Registers are a part of the CPU, but act as ultra-fast, tiny memory locations.

- They are the absolute fastest storage locations in a computer.
- The ALU can ONLY process data if it is currently sitting inside a Register.
- Types of Registers:
 - **IR (Instruction Register):** Holds the current instruction being executed.
 - **PC (Program Counter):** Holds the address of the *next* instruction to be fetched.
 - **Accumulator:** Holds intermediate arithmetic results.



FUNCTIONAL BLOCK DIAGRAM





STEP-BY-STEP DATA FLOW (EXAMPLE: 7 + 5)

Let's trace how the computer processes the calculation **7 + 5**:

- **Step 1 (Input):** User types '7 + 5' via the keyboard. Data is converted to binary.
- **Step 2 (Memory):** The values '7', '5' and the instruction 'ADD' are stored in Main Memory.
- **Step 3 (Control Unit):** Fetches the 'ADD' instruction, decodes it, and sends signals to route 7 and 5 to the CPU registers.
- **Step 4 (ALU):** Receives binary for 7 and 5, performs logic gate addition, and outputs the result.
- **Step 5 (Memory):** Result '12' is temporarily sent back to memory.
- **Step 6 (Output):** Control unit routes the result to the monitor to display '12'.



REAL-LIFE ANALOGY

The Restaurant Analogy

- **Input Unit:** The Waiter taking your specific order from the menu.
- **Memory Unit:** The kitchen order ticket rail (holds pending/active orders).
- **Control Unit:** The Head Chef reading the ticket, shouting commands, and timing
- **ALU:** The line cooks actually chopping vegetables and cooking (doing the real work).
- **Output Unit:** The waiter delivering the finished, plated food to your table.



CLASSROOM ACTIVITY

Interactive Tracing

Activity: Trace the exact data and control path for a "Save Document" command.

Questions for the class:

- Where is the document residing before you click save? (RAM/Primary Memory)
- Which unit detects your mouse click on the 'Save' icon? (Input Unit)
- Which unit directs the physical data transfer? (Control Unit)
- Where does the final data end up? (Secondary Memory / Hard Drive)



DAILY QUIZ & WEEKLY ASSIGNMENT

DAILY QUIZ

Which unit acts as the "nervous system", generating timing signals to

A. Arithmetic Logic Unit

B. Memory Unit

C. Control Unit

D. Output Unit



WEEKLY ASSIGNMENT 1

1. Draw the block diagram of a computer system in detail. Differentiate between the exact roles of the ALU and the Control Unit.
2. Explain the necessity of CPU Registers.
- 3.



MULTIPLE CHOICE QUESTIONS

Q1 Which architecture shares physical memory and buses for both data and instructions?

A. Von Neumann

B. Harvard Architecture

C. RISC Architecture

D. None of the above

Q2 What is the main limitation resolved by the Harvard Architecture?

A. High cost of design

B. Memory Bottleneck

C. High power use

D. Complex decoding

Q3 Which storage system has physical separate signals for instructions & data?

A. Von Neumann

B. Harvard Architecture

C. Unified Cache System

D. Accumulator Machine

Q4 Which system allows execution of instruction while fetching next data?

A. Non-Pipelined VN

B. Simple Accumulator

C. Harvard System

D. None of the above



TOPIC LEARNING LINK



TOPIC LEARNING LINK

Title: Basic Functional Units of a Computer

Resource: Recommended Engineering Tutorials

[https://www.youtube.com/watch?
v=DKGZlaPIVLY](https://www.youtube.com/watch?v=DKGZlaPIVLY)



TOPIC 3: VN & HARVARD ARCHITECTURES

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To explore the structural differences between Von Neumann and Harvard physical architectures.

Topic Outcome: Students will be able to distinguish between shared and separate memory structures, draw their respective bus diagrams, and define the infamous Von Neumann Bottleneck.



THE STORED PROGRAM CONCEPT

Before 1945, computers were hard-wired to perform a single task. To change the program, engineers had to physically rewire the machine!

John von Neumann's breakthrough (1945):

The "Stored Program Concept". Instructions (the program software) and Data should be stored electronically in the SAME memory device.

This meant a computer could be easily reprogrammed simply by loading different instruction values into memory, creating the modern general-purpose computer.

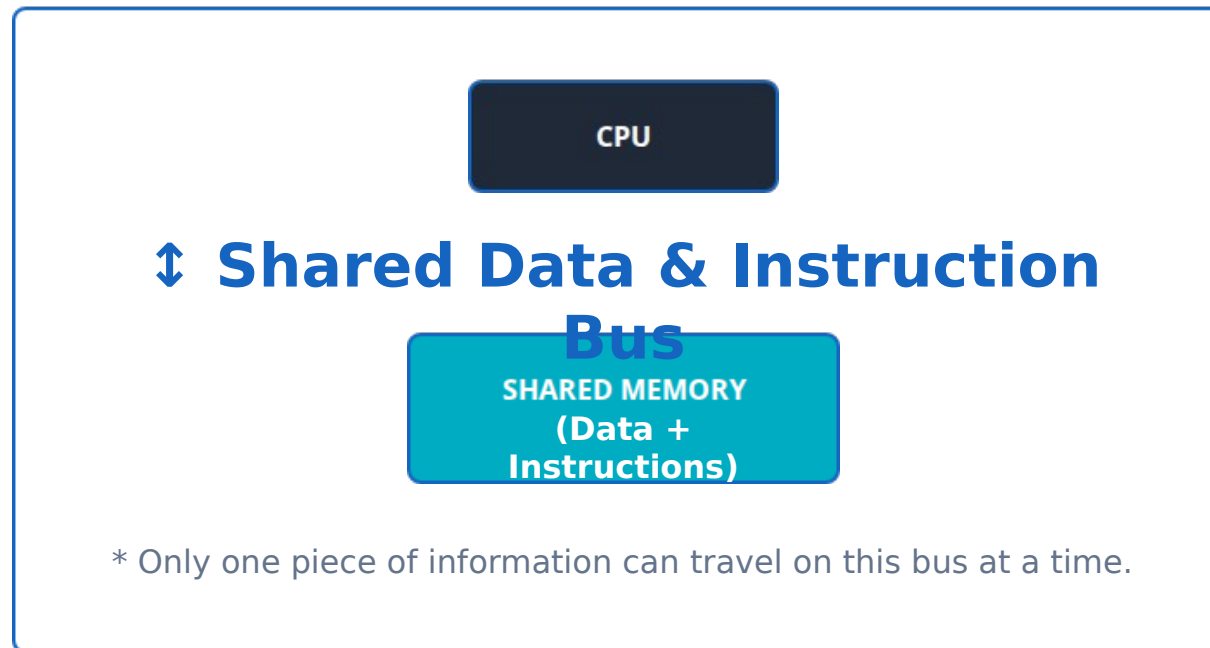


VON NEUMANN ARCHITECTURE

Based on the Stored Program Concept, this architecture features:

- A single, **shared memory** block for both Data and Instructions.
- A single **shared bus system** (the physical wires routing data).
- The CPU fetches either an instruction OR data from memory at one time, but *cannot do both at the same time*.
- **Advantage:** Simple to design and cheaper hardware.
- **Disadvantage:** Suffers from the "Von Neumann Bottleneck".

VON NEUMANN TECHNICAL DIAGRAM





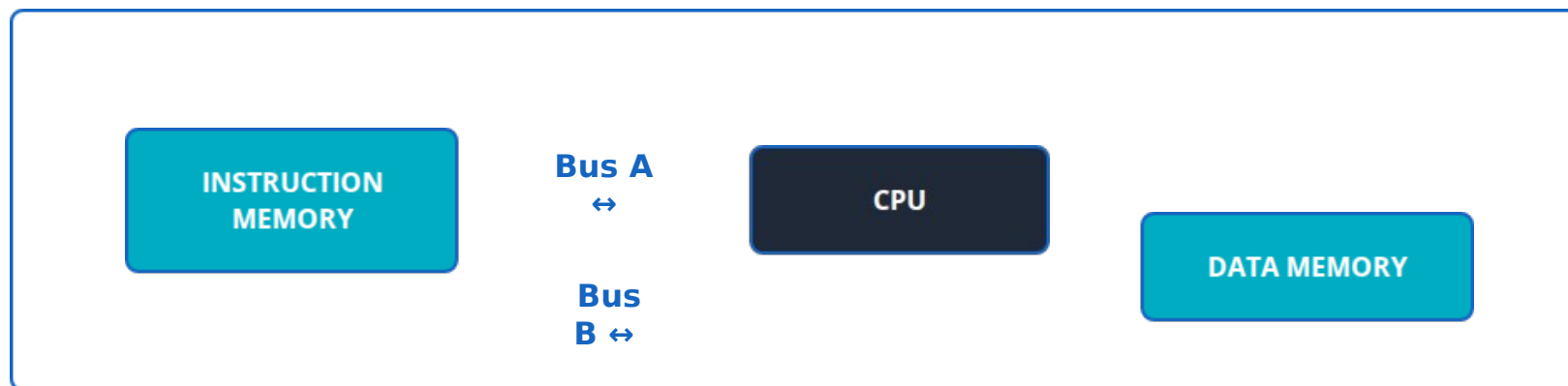
HARVARD ARCHITECTURE

Developed concurrently at Harvard University, this architecture takes a different approach to bypass the bottleneck:

- Features **separate physical memory modules** for Instructions and Data.
- Features **separate hardware buses** for Instructions and Data.
- The CPU can fetch an instruction AND read/write data *simultaneously* (Parallelism).
- **Advantage:** Much faster execution speed.
- **Disadvantage:** More complex routing and expensive hardware manufacturing.



HARVARD TECHNICAL DIAGRAM



* CPU can talk to both memories simultaneously using separate buses.



REAL-LIFE ANALOGY

Von Neumann: One-Lane Bridge

Imagine a single-lane bridge (the shared bus). Cars (data) and trucks (instructions) must wait their turn to cross. If the CPU processes data faster than the bridge allows traffic, the CPU sits idle. This traffic jam is the **Von Neumann Bottleneck**.

Harvard: Two-Lane Bridge

Harvard architecture is a two-lane bridge. Cars (data) and trucks (instructions) have their own dedicated, separate lanes. They can cross simultaneously, completely eliminating the traffic jam.



COMPARISON TABLE

Feature	Von Neumann Architecture	Harvard Architecture
Memory Structure	Shared (Data + Instructions)	Separate (Data & Instructions)
Bus System	Single shared bus	Separate data & instruction buses
Execution Speed	Slower (Sequential fetch)	Faster (Parallel fetch)
Hardware Cost	Simpler, cheaper	Complex, expensive
Primary Usage	General-purpose PCs (Main RAM)	Microcontrollers, DSPs, CPU Caches



EXAM POINTS & DAILY QUIZ

Important Exam Point:

Modern CPUs actually use a *hybrid* approach! They use Von Neumann for external Main RAM, but Harvard internally for L1 Instruction and Data Caches.

DAILY QUIZ

The Von Neumann bottleneck is primarily caused by:

A. Slow CPU clock

B. A shared bus

C. Separate memories

D. Fast I/O devices



MCQ PRACTICE

STEP 2: MULTIPLE CHOICE QUESTIONS

1. What is the radix (base) of the Octal number system?

A. 2

B. 10

C. 8 (Correct)

D. 16

2. Which binary value represents Decimal 10?

A. 1100

B. 1010 (Correct)

C. 1001

D. 0111

3. In Hexadecimal, what is the value of 'F' in decimal?

A. 10

B. 12

C. 14

D. 15 (Correct)

4. Converting Binary 111 to Octal results in:

A. 7 (Correct)

B. 6

C. 5

D. 8



TOPIC LEARNING LINK



TOPIC LEARNING LINK

Title: Von Neumann vs Harvard Architecture Visualized

Resource: Standard Architecture Lectures

[https://youtu.be/YiZy6vfgtHU?
si=ymZZhtiPngaR3isF](https://youtu.be/YiZy6vfgtHU?si=ymZZhtiPngaR3isF)



TOPIC 4: NUMBER SYSTEMS & CODES

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To comprehensively understand the mathematical systems used by computers to represent physical data.

Topic Outcome: Students will be able to define base/radix and seamlessly convert between Decimal, Binary, Octal, and Hexadecimal systems.



INTRODUCTION: BASE, RADIX & POSITION

Number System: A mathematical methodology to represent numbers logically.

Base or Radix: The absolute total number of unique digits used in the system.

Positional Value: The value of a digit strictly depends on its position (e.g., Units, Tens, Hundreds).

Example in Decimal

(Base 10): 345 is mathematically represented as:

$$3 \times 10^2 + 4 \times 10^1 + 5 \times 10^0$$

$$300 + 40 + 5 = 345$$



DECIMAL & BINARY SYSTEMS

Decimal System (Base 10)

- **Digits:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- The standard system used by humans (derived from our 10 fingers).
- Used for final data output to the user.

Binary System (Base 2)

- **Digits:** 0 and 1 (Called Bits).
- The fundamental language of the computer.
- Why? Computers use billions of microscopic electronic transistors that act as switches.
- Switches have two highly reliable physical states: OFF (0V) and ON (5V).



OCTAL & HEXADECIMAL SYSTEMS

Octal System (Base 8):

- **Digits:** 0, 1, 2, 3, 4, 5, 6, 7.
- Used to represent large binary numbers compactly (combining exact groups of 3 bits).

Hexadecimal System (Base 16):

- **Digits:** 0-9 and A-F. (Combines exact groups of 4 bits).
- A=10, B=11, C=12, D=13, E=14, F=15.
- Highly utilized in programming (MAC addresses, IPv6 routing, Memory addresses, Web RGB color codes).



COMPARISON OF NUMBER SYSTEMS

Number System	Base (Radix)	Valid Digits	Primary Computing Application
Binary	2	0, 1	Core machine language and logic gates
Octal	8	0 to 7	Unix file permissions, legacy systems
Decimal	10	0 to 9	Human interaction and output displays
Hexadecimal	16	0-9, A-F	Memory addresses, IPv6, Color codes



DECIMAL TO BINARY & REVERSE

Dec to Bin (Repeated Div by 2):

Convert 13 to Binary:

$13 / 2 = 6$, Rem 1 (LSB)

$6 / 2 = 3$, Rem 0

$3 / 2 = 1$, Rem 1

$1 / 2 = 0$, Rem 1 (MSB)

Read bottom to top: **(1101)₂**

Bin to Dec (Positional Weight):

Convert (1011)₂ to Decimal:

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$8 + 0 + 2 + 1 = \mathbf{(11)_{10}}$$



DECIMAL TO OCTAL & REVERSE

Dec to Octal (Repeated Div by 8):

Convert 37 to Octal:

$37 / 8 = 4$, Rem 5

$4 / 8 = 0$, Rem 4

Read bottom to top: **(45)₈**

Octal to Dec (Positional Weight):

Convert (45)₈ to Decimal:

$$4 \times 8^1 + 5 \times 8^0$$

$$32 + (5 \times 1) = \mathbf{(37)_{10}}$$



DECIMAL TO HEXADECIMAL & REVERSE

Dec to Hex (Repeated Div by 16):

Convert 42 to Hexadecimal:

$42 / 16 = 2$, Rem 10 (Replace with A)

$2 / 16 = 0$, Rem 2

Read bottom to top: **(2A)16**

Hex to Dec (Positional Weight):

Convert (2A)16 to Decimal:

$$2 \times 16^1 + A \times 16^0$$

$$(2 \times 16) + (10 \times 1)$$

$$32 + 10 = \mathbf{(42)10}$$



BINARY TO OCTAL & HEXADECIMAL

Binary to Octal (3-Bit Grouping from Right to Left):

Convert $(110101)_2$ to Octal:

1. Group: $(110) (101)$
2. Convert groups directly: 6 5
3. Result: **$(65)_8$**

Binary to Hexadecimal (4-Bit Grouping):

Convert $(11010110)_2$ to Hexadecimal:

1. Group: $(1101) (0110)$
2. Convert groups: 13 6 (Replace 13 with D)
3. Result: **$(D6)_{16}$**



CONVERSION SOLVED EXAMPLES

Example 1: Convert $(F3)_{16}$ to Binary.

- Write each hex digit as exactly 4 bits.
- $F = 15 = 1111$
- $3 = 0011$
- Combine: $(11110011)_2$

Example 2: Convert $(10110)_2$ to Hexadecimal.

- Group by 4 from the right: $(0001) (0110)$ **Pad with left zeros!*
- Convert: 1 6
- Result: $(16)_{16}$

DAILY QUIZ & WEEKLY ASSIGNMENT

DAILY QUIZ

What is the exact decimal value
of hexadecimal 'F'?

A. 14

B. 15

C. 16

D. 10



WEEKLY ASSIGNMENT 2

- 1 Convert (255)₁₀ to Binary, Octal,
and Hexadecimal.
- 2 Convert (1101101)₂ to
Hexadecimal.
- 3 Why do programmers prefer
hexadecimal over pure binary?



DAILY QUIZ: PRACTICE MCQS

QUESTION 1

What is the binary representation of the decimal number 12?

A. 1010

B. 1100

C. 1110

D. 1011

QUESTION 2

Which of the following represents hexadecimal value 'A' in decimal?

A. 10

B. 11

C. 12

D. 15

QUESTION 3

How many bits are there in a single byte?

A. 4 Bits

B. 8 Bits

C. 16 Bits

D. 32 Bits

QUESTION 4

Which logic gate outputs 1 only when both inputs are 1?

A. OR Gate

B. AND Gate

C. NOT Gate

D. XOR Gate



TOPIC LEARNING LINK



TOPIC LEARNING LINK

Title: Number System Conversions Tutorial

Resource: Recommended Engineering Tutorials

[https://youtu.be/Mxjjqq3B6JU?
si=AvolH1-j-zoQXVrb](https://youtu.be/Mxjjqq3B6JU?si=AvolH1-j-zoQXVrb)



TOPIC 5: BINARY ARITHMETIC

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To perform fundamental arithmetic operations directly on binary numbers at the logic level.

Topic Outcome: Students will master the strict rules of binary addition and subtraction, comprehensively understanding carry and borrow mechanisms used by the ALU.



BINARY ADDITION RULES

Adding binary bits follows 5 simple foundational rules:

$0 + 0 = 0$ (Sum = 0, Carry = 0)
 $0 + 1 = 1$ (Sum = 1, Carry = 0)
 $1 + 0 = 1$ (Sum = 1, Carry = 0)
 $1 + 1 = 10$ (Sum = 0, Carry = 1)
 $1 + 1 + 1 = 11$ (Sum = 1, Carry = 1)

Concept: The 'Carry' is always shifted left to the next higher positional column, exactly like decimal addition.



BINARY ADDITION SOLVED EXAMPLES

Example 1: Add $(101)_2$ and $(011)_2$

$$\begin{array}{r} 1 \ 1 \ 1 \quad (\text{Carries}) \\ 1 \ 0 \ 1 \quad (\text{Decimal } 5) \\ + 0 \ 1 \ 1 \quad (\text{Decimal } 3) \\ \hline 1 \ 0 \ 0 \ 0 \quad (\text{Decimal } 8) \end{array}$$

Col 1 (Right): $1+1 = 10$ (Write 0, carry 1)

Col 2: $1(\text{carry}) + 0 + 1 = 10$ (Write 0, carry 1)

Col 3: $1(\text{carry}) + 1 + 0 = 10$ (Write 0, carry 1)

Col 4 (Left): Bring down carry 1 $\rightarrow 1000$



BINARY SUBTRACTION RULES

Subtracting binary bits requires borrowing from a higher power:

$$0 - 0 = 0$$

$$1 - 0 = 1$$

$$1 - 1 = 0$$

$$0 - 1 = 1 \text{ (Requires a Borrow of 1 from the next higher bit)}$$

The Borrow Concept:

When you borrow 1 from the next left column, the current 0 becomes 10 in binary (which is 2 in decimal). Thus, you are mathematically calculating $2 - 1 = 1$.



BINARY SUBTRACTION EXAMPLES

Example: Subtract $(011)_2$ from $(110)_2$

$$\begin{array}{r} 110 \text{ (Decimal 6)} \\ - 011 \text{ (Decimal 3)} \\ \hline 011 \text{ (Decimal 3)} \end{array}$$

Col 1: $0 - 1$. Borrow 1. Becomes $10 - 1 = 1$.

Col 2: The 1 became 0. Now $0 - 1$. Borrow 1. Becomes $10 - 1 = 1$.

Col 3: The 1 became 0. $0 - 0 = 0$.

Verification: $6 - 3 = 3$.



DAILY QUIZ & TOPIC LINK

DAILY QUIZ

In digital binary addition, the operation $1 + 1 + 1$ yields:

A. Sum 0 with carry 1

B. Sum 1 with carry 1

C. Sum 0 with carry 0

TOPIC LEARNING LINK

Video: Binary Addition and

Subtraction With Borrowing

<https://youtu.be/4cjs2GrOf6Y?si=zB3U4fVOUPIC5KxE>



BINARY ARITHMETIC QUIZ

Q1 What is the sum of $(1010)_2$ and $(1101)_2$ in binary?

A. $(10111)_2$

B. $(10111)_2$ ✓ (Correct)

C. $(11011)_2$

D. $(11111)_2$

Explanation: $10 + 13 = 23$ in decimal, which is represented as 10111 in binary.

Q3 What is the binary multiplication of $(101)_2$ by $(11)_2$?

A. $(1110)_2$

B. $(1001)_2$

C. $(1111)_2$ ✓ (Correct)

D. $(1101)_2$

Explanation: $5 \times 3 = 15$ in decimal, which is represented as 1111 in binary.

Q2 Subtract $(0110)_2$ from $(1101)_2$. What is the result?

A. $(0111)_2$ ✓ (Correct)

B. $(1001)_2$

C. $(0101)_2$

D. $(0011)_2$

Explanation: $13 - 6 = 7$ in decimal, which is represented as 0111 in binary.

Q4 Divide $(1100)_2$ by $(11)_2$. What is the quotient?

A. $(101)_2$

B. $(100)_2$ ✓ (Correct)

C. $(010)_2$

D. $(110)_2$

Explanation: $12 \div 3 = 4$ in decimal, which is represented as 100 in binary.



TOPIC 6: BCD, GRAY CODE & ASCII

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To understand specialized binary encoding standards used for data integrity and human-readable text.

Topic Outcome: Students will be able to encode decimal to BCD, perform logic-based Binary-to-Gray conversions, and explain how English text is stored in RAM using ASCII.



BCD (BINARY CODED DECIMAL)

Rule: Each single decimal digit is represented by exactly 4 bits. Binary values 1010 to 1111 are totally invalid in BCD.

Decimal Digit	Pure Binary (4-bit)	BCD Representation
0	0000	0000
5	0101	0101
9	1001	1001
10	1010 (Pure)	0001 0000 (BCD)
12	1100 (Pure)	0001 0010 (BCD)



GRAY CODE

Gray Code is an unweighted binary code.

One-Bit Change Property: Only ONE single bit changes state between any two consecutive numbers. This greatly reduces mechanical errors in sensors.

Real-World Applications:

- Rotary Encoders (Volume knobs, robotic arms).
- Position Sensors in automotive systems.
- Industrial Control systems.
- Digital Communication (Error correction).



BINARY TO GRAY CODE CONVERSION

Conversion Rule (using XOR Logic):

- 1 MSB (left-most bit) of Gray is identical to MSB of Binary.
- 2 Next Gray bit = XOR of current binary bit and previous binary bit.
- .

Solved Example: Convert Binary 1011 to Gray.

- Binary = 1 0 1 1
- $G_3 = B_3 = 1$
- $G_2 = 1 \text{ XOR } 0 = 1$
- $G_1 = 0 \text{ XOR } 1 = 1$
- $G_0 = 1 \text{ XOR } 1 = 0$
- Final Gray Code = **1110**



ASCII CODES

ASCII: American Standard Code for Information Interchange.

It is a standard that perfectly connects physical keys on a keyboard to electrical binary values computers can store in memory.

Key ASCII Decimal Values to Memorize:

- Uppercase 'A' to 'Z' = 65 to 90
- Lowercase 'a' to 'z' = 97 to 122
- Numbers '0' to '9' = 48 to 57
- Spacebar = 32



EXAMPLE: STORING TEXT IN RAM

Question: How is the physical word 'CAT' actually stored in silicon RAM?

```
Step 1: 'C' -> ASCII 67 -> Binary 01000011  
Step 2: 'A' -> ASCII 65 -> Binary 01000001  
Step 3: 'T' -> ASCII 84 -> Binary 01010100
```

RAM holds: 01000011 01000001
01010100

This demonstrates how human textual data is fundamentally just physical binary electrical signals inside the machine.

Computer Organization | Unit 1 | Course Outcome 1

DAILY QUIZ & WEEKLY ASSIGNMENT

DAILY QUIZ

What is the standard ASCII decimal value of uppercase 'A'?

A. 97

B. 48

C. 65

D. 32



WEEKLY ASSIGNMENT

- 1 Convert Decimal 45 to BCD format.
- 2 Convert Binary 1100 directly to
Write Code decimal values.
- 3 Write your own first name in ASCII
decimal values.



TOPIC 7: BOOLEAN ALGEBRA

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To rigorously learn the mathematics governing digital logic circuits.

Topic Outcome: Students will be able to apply Boolean theorems to simplify complex logic expressions, directly reducing manufacturing hardware costs.



INTRODUCTION TO BOOLEAN ALGEBRA

Invented by mathematician George Boole in 1854. It is the specific algebra of truth values:

- **0 = FALSE / LOW Voltage / OFF state.**
- **1 = TRUE / HIGH Voltage / ON state.**

Boolean Variables: Variables like A, B, X, Y that can ONLY hold 0 or 1.

Basic Core Operations:

- **AND** (Multiplication / Dot notation: $A \cdot B$)
- **OR** (Addition / Plus notation: $A + B$)
- **NOT** (Inversion / Bar or Prime notation: A')



BOOLEAN AXIOMS

Identity & Null Laws

Identity Law:

$$A + 0 = A$$

$$A \cdot 1 = A$$

Null Law:

$$A + 1 = 1$$

$$A \cdot 0 = 0$$

Idempotent & Complement

Idempotent Law:

$$A + A = A$$

$$A \cdot A = A$$

Complement Law:

$$A + A' = 1$$

$$A \cdot A' = 0$$



BOOLEAN LAWS

Commutative & Associative

Commutative:

$$A + B = B + A$$

$$A \cdot B = B \cdot A$$

Associative:

$$A + (B + C) = (A + B) + C$$

Distributive & Absorption

Distributive:

$$A (B + C) = AB + AC$$

$$A + BC = (A + B) (A + C)$$

Absorption:

$$A + AB = A$$



DE MORGAN'S THEOREMS

Extremely critical for digital circuit design and gate conversion.

Theorem 1: (The complement of a sum is equal to the product of the complements)

$$(A + B)' = A' \cdot B'$$

Theorem 2: (The complement of a product is equal to the sum of the complements)

$$(A \cdot B)' = A' + B'$$

Memory trick: "Break the line, change the sign."



SOLVED SIMPLIFICATION EXAMPLES

Example 1: Simplify $Y = AB + AB'$

- Step 1: Factor out A $\rightarrow Y = A(B + B')$
- Step 2: Apply Complement Law ($B + B' = 1$) $\rightarrow Y = A(1)$
- Step 3: Apply Identity Law $\rightarrow \mathbf{Y = A}$

Hardware Impact: A 2-gate circuit is reduced to just a direct piece of wire!

Example 2: Simplify $Y = A + A'B$

- Step 1: Distributive Law $\rightarrow (A + A')(A + B)$
- Step 2: Complement Law $\rightarrow (1)(A + B)$
- Step 3: Identity Law $\rightarrow \mathbf{Y = A + B}$



DAILY QUIZ & TOPIC LINK

DAILY QUIZ

According to De Morgan's theorem,
the expression $(A \cdot B)'$ is strictly equal

A. $A + B'$

B. $A' \cdot B'$

C. $A + B$

D. $(A+B)'$

TOPIC LEARNING LINK

Search: Boolean Algebra

Simplification Rules

[https://youtu.be/pFxDHrUut6w?
si=jEXHj78uk8fUsKQL](https://youtu.be/pFxDHrUut6w?si=jEXHj78uk8fUsKQL)



BOOLEAN ALGEBRA QUIZ

QUESTION 1

Which law is represented by the equation $A + (B \cdot C) = (A + B) \cdot (A + C)$?

- A. Associative Law
- C. Commutative Law

QUESTION 2

According to De Morgan's Theorem, what is the equivalent of $(A + B)'$?

- B. $A' + B'$
- C. $(A \cdot B)'$

QUESTION 3

What is the simplified output of the Boolean expression: $A \cdot (A + B)$?

- A. B
- C. $A \cdot B$

QUESTION 4

Which Boolean law states that the term $A + A'$ is always equal to 1?

- A. Identity Law
- B. Idempotent Law



TOPIC 8: LOGIC GATES

STEP 1: TOPIC OBJECTIVE & OUTCOME

Topic Objective: To study electronic physical circuits that make logical decisions based on voltage inputs.

Topic Outcome: Students will be able to draw exact logic symbols, write truth tables, and use universal gates to build larger digital processor circuits.



BASIC GATES: AND, OR, NOT

AND Gate (ALL)

Output is HIGH only if
ALL inputs are HIGH.
Expression: $Y = A \cdot B$

0 0 $\rightarrow$ 0

1 1 $\rightarrow$ 1

OR Gate (ANY)

Output HIGH if ANY
input is HIGH.
Expression: $Y = A + B$

0 0 $\rightarrow$ 0

0 1 $\rightarrow$ 1

NOT Gate (OPPOSITE)

Inverts the input signal.
Expression: $Y = A'$

0 $\rightarrow$ 1

1 $\rightarrow$ 0



UNIVERSAL GATES: NAND & NOR

Why "Universal"? You can physically build ANY logic circuit using ONLY NAND or ONLY NOR gates. Highly used in manufacturing CPUs.

NAND Gate (NOT AND):

- Exact inverse of the AND gate. $Y = (A \cdot B)'$
- Output is LOW only when all inputs are HIGH.

NOR Gate (NOT OR):

- Exact inverse of the OR gate. $Y = (A + B)'$
- Output is HIGH only when all inputs are LOW.



XOR AND XNOR GATES

XOR Gate (Different)

Exclusive-OR.

Output is HIGH if inputs are DIFFERENT.

$$Y = A \oplus B = A'B + AB'$$

0 0 → 0

0 1 → 1

1 0 → 1

1 1 → 0

XNOR Gate (Same)

Exclusive-NOR.

Output is HIGH if inputs are the SAME.

$$Y = A \odot B = AB + A'B'$$

0 0 → 1

0 1 → 0

1 0 → 0

1 1 → 1



TRUTH TABLES & MEMORY TRICKS

Inputs (A, B)	XOR (Different) Output	XNOR (Same) Output
0, 0	0	1
0, 1	1	0
1, 0	1	0
1, 1	0	1

Memory Tricks for Exams:

AND = ALL

OR = ANY

NOT = OPPOSITE

XOR = Detects Difference (Inequality detector)

XNOR = Detects Sameness (Equality detector)



REAL-LIFE ANALOGIES & DIAGRAMS

- **AND Gate:** Two electrical switches placed in SERIES. Both must be closed for the bulb to turn on.
- **OR Gate:** Two switches placed in PARALLEL. Closing either switch turns the bulb on.
- **XOR Gate:** Staircase wiring (two switches controlling one bulb from different floors).

Gate Symbols:

AND: Straight back, curved front (D-shaped).

OR: Curved back, pointed front (Shield-like).

NOT: Triangle with a small circle (bubble).

Engineering Standard: A bubble ALWAYS means 'Inversion' (NOT).



DAILY QUIZ & TOPIC LINK

DAILY QUIZ

Which specific logic gate produces a HIGH output only when both inputs are different?

A. AND

B. OR

C. XOR

D. XNOR

TOPIC LEARNING LINK

Try it: Logic Gates Interactive Simulator (CircuitVerse)



DAILY REVISION: MCQ QUIZ

QUESTION 1

Which logic gate produces a HIGH output only when both inputs are different?

A. AND

B. OR

C. XOR

D. XNOR

QUESTION 2

What is the key architectural characteristic of combinational circuits?

A. Internal Memory

B. Feedback Loops

C. No Memory

D. Clock Triggered

QUESTION 3

Which of the following represents a data routing combinational circuit?

A. Multiplexer

B. Register

C. Flip-Flop

D. Counter

QUESTION 4

In daily electrical wiring, staircase circuit control operates like which gate?

A. AND Gate

B. OR Gate

C. XOR Gate

D. NAND Gate



TOPIC 9: COMBINATIONAL CIRCUITS

Definition: A digital logic circuit in which the output strictly depends ONLY on the PRESENT combination of inputs at that exact microscopic moment.

Key Architectural Characteristic: NO MEMORY.

It cannot store data. It does not have any feedback loops.

Rule: Present Input → Present Output.

n Inputs → [COMBINATIONAL LOGIC CIRCUIT] → m Outputs



EXAMPLES & DAILY QUIZ

Common Processor Hardware Examples:

- **Arithmetic:** Adders, Subtractors (Inside ALU).
- **Data Routing:** Multiplexers (MUX), Demultiplexers (DEMUX).
- **Code Converters:** Encoders, Decoders (Inside Control Unit).

Combinational circuits exclusively possess which characteristic?

A. Internal memory

B. Feedback loops

C. Output depends ONLY on present inputs



TOPIC 10: HALF ADDER

Definition: A foundational combinational circuit that performs the arithmetic addition of two single-bit binary numbers.

Inputs: A, B (The two bits to be added)

Outputs: SUM, CARRY

A	B	SUM	CARRY
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



HALF ADDER LOGIC & LIMITATION

Boolean Expressions:

$$\text{SUM} = A \oplus B \text{ (XOR Gate)}$$

$$\text{CARRY} = A \cdot B \text{ (AND Gate)}$$

Major Limitation: A Half Adder can ONLY add two single bits. It absolutely CANNOT accept a carry generated from a previous lower-order bit addition. Therefore, it cannot add multi-bit numbers by itself.

The SUM output logic of a Half Adder is perfectly identical to the output of a(n):

XOR Gate



TOPIC 11: FULL ADDER

Definition: An advanced combinational circuit that adds three 1-bit binary numbers (A, B,

Inputs: A, B, Cin

Outputs: SUM, Cout

$$\text{SUM} = A \oplus B \oplus \text{Cin}$$

$$\text{Cout} = AB + B\text{Cin} + A\text{Cin}$$

FULL ADDER IMPLEMENTATION

A Full Adder is commonly constructed by cascading two Half Adders and an OR gate.

- 1 Half Adder 1 adds A and B → Generates Sum1 and Carry1.
- 2 Half Adder 2 adds Sum1 and Cin → Generates Final SUM and Carry2.
- 3 OR gate evaluates Carry1 and Carry2 → Generates Final Cout.
- .

A standard full adder can be built efficiently using:

A. 2 Half Adders + 1 OR gate



TOPIC 12: MULTIPLEXER (MUX)

Definition: A digital logic switch. Takes MANY inputs and selects ONE to pass to the output.

Analogy: TV Channel Selector. Many channels arrive (inputs), but the remote (selection lines) chooses only ONE to show on the screen.

Formula: For 2^n inputs, n selection lines are required.

Example: A 4:1 MUX has 4 inputs (I0, I1, I2, I3) and 2 selection lines (S1, S0).



MUX APPLICATIONS & QUIZ

Applications: Internal data routing inside CPUs (ALU bus selection), Telephone networks, Parallel to serial converters.

DAILY QUIZ

How many hardware selection lines are strictly required to operate an

A. 1

B. 2 8:1 MUX?

C. 3

D. 4



TOPIC 13: DEMULTIPLEXER (DEMUX)

Definition: Takes ONE single input and precisely distributes it to MANY possible outputs.

Analogy: A courier sorting facility. One truck arrives (input), and the sorter logic (selection lines) routes each package to the correct local delivery van (many outputs).

Memory Trick:

MUX = MANY TO ONE

DEMUX = ONE TO MANY



DEMUX COMPARISON & QUIZ

Feature	Multiplexer (MUX)	Demultiplexer (DEMUX)
Function	Data Selector	Data Distributor
	Many to One (	One to Many (
	$2^n : 1$	$1 : 2^n$
	2	1
	:	:
Ratio	n	2
	A DEMUX strictly has:	n
	:	
	1	
	)	)
Inputs/Outputs	Many Inputs, 1 Output	1 Input, Many Outputs



WHOLE UNIT MCQS & GLOSSARY

UNIT MCQs

- Q1. Which base is hexadecimal? (A) 2 (B) 8 (C) 16 → C
- Q2. Universal gate? (A) AND (B) NAND (C) OR → B
- Q3. $(A+B)' = ?$ (A) $A'B'$ (B) $A'+B' \rightarrow A$
- Q4. CPU = ALU + ? (A) Mem (B) CU (C) I/O → B

GLOSSARY

- **Base/Radix:** Total unique digits in a system.
- **Bit:** Binary Digit (0 or 1).
- **ALU:** Arithmetic Logic Unit.
- **Combinational:** Logic with zero memory storage.



WHOLE UNIT MCQS

- Q1** Which number system base represents Hexadecimal notation?
(A) Base-2 (B) Base-8 (C) Base-16 (D) Base-10

Correct Answer: (C) Base-16 [cite: 105]

Hexadecimal uses 16 unique symbols (0-9 and A-F) [cite: 105].

- Q2** Which of the following logic gates is classified as a universal gate?
(A) AND (B) NAND (C) OR (D) XOR

Correct Answer: (B) NAND [cite: 105]

NAND and NOR gates can implement any Boolean function [cite: 105, 107].

- Q3** According to De Morgan's Laws, what is the equivalent of $(A + B)'$?
(A) $A'B'$ (B) $A' + B'$ (C) AB (D) $A' + B$

Correct Answer: (A) $A'B'$ [cite: 105]

The complement of a sum is the product of the complements [cite: 105].

- Q4** A central processing unit (CPU) consists of ALU and:
(A) Memory (B) Control Unit (CU) (C) Input/Output (D) Cache

Correct Answer: (B) CU [cite: 105]

The CPU fundamentally integrates the ALU and the Control Unit [cite: 105].



EXAMINATION QUESTIONS

Important Short Questions (2 Marks):

- 1. Define Computer Organization vs Architecture.
- 2. Convert $(55)_{10}$ to binary.
- 3. Why are NAND and NOR called universal gates?

Important Long Questions (5-10 Marks):

- 1. Explain Von Neumann vs Harvard Architecture with block diagrams.
- 2. Draw and formally explain the logic diagram and truth table of a Full Adder.
- 3. Simplify the Boolean expression $Y = A'B + AB + A'B'$ using axioms and theorems.

THANK YOU

COMPUTER ORGANIZATION

UNIT 1

Presented By: Bibrata Sarkar

Course Outcome 1

